IIT Poker Club: Multi-Table Tournament (MTT) Pokerbot Submission Guide

Poker Club IIT

Submission Due: April 23, 2026

1 Submission Guidelines

To enter the tournament, participants must adhere to the following protocols:

- **Deadline:** Submissions must be received by **April 23, 2026**.
- **Limit:** Exactly one (1) submission is allowed per team, up to 4 members per team.
- **Method:** Upload your `.py` script here when you are ready to submit.
- **Testing:** Participants are required to test their bots using the built-in visualizer and against their own test bots prior to submission.

2 Tournament Structure & Scoring

The winner will be determined based on the aggregate performance across **10 simulations** of a Multi-Table Tournament (MTT).

- Every submitted pokerbot will participate in each simulation.
- Points are awarded to the Top 3 finishers of each simulation as follows:

Placement	Point Distribution
1st Place	50
2nd Place	30
3rd Place	20

To clarify how the overall winner is chosen, consider a simplified example. Suppose four bots (Bot 1, Bot 2, Bot 3, and Bot 4) compete over three simulations (for brevity, though the full tournament runs ten):

Bot	Sim 1 (Points)	Sim 2 (Points)	Sim 3 (Points)	Total Points
Bot 1	1st (50)	2nd (30)	—(0)	80
Bot 2	2nd (30)	1st (50)	1st (50)	130
Bot 3	3rd (20)	3rd (20)	2nd (30)	70
Bot 4	—(0)	—(0)	3rd (20)	20

Table 1: Illustrative example of points scoring and total weighted calculation

In this illustrative example, Bot 2 would be declared the winner with a total of 130 points. The final tournament will apply this methodology over ten complete simulations to determine the definitive champion.

3 Technical Environment

The tournament utilizes the **MTT Pokerbot Engine**. You can access the source code and engine documentation here:
https://github.com/Alex-Addison/MTT-Pokerbot-Engine

3.1 Allowed Libraries

For security and fairness, bots are restricted to the following libraries:

- `math`, `random`, `numpy`, `pandas`

Note: Attempts to import unauthorized modules (e.g., `os`, `sys`) will result in the bot being blocked by the engine.

3.2 Bot Logic Template

Your bot must contain a `get_action` function. This function receives a `game_state` dictionary and must return a decision within **500ms**.

Listing 1: Example Bot Action

```
def get_action(game_state):
    # Determine action based on game_state (hole_cards, pot_size, etc.)
    # Options: ('fold', 0), ('call', 0), or ('raise', X)
    return ('call', 0)
```

4 Default Configuration

The following settings in `config.json` represent the official tournament environment:

- **Starting Stack:** 1500 chips.
- **Decision Timeout:** 500ms (Half-second).
- **Table Size:** 9-max.
- **Blinds:** Progressive schedule (defined in engine config).

5 Instructions for Local Setup

1. **Clone Repo:** `git clone https://github.com/Alex-Addison/MTT-Pokerbot-Engine.git`
2. **Install Requirements:** `python -m pip install -r requirements.txt`
3. **Run Simulation:** Place your bot in the `/players` folder and run `python main.py`.

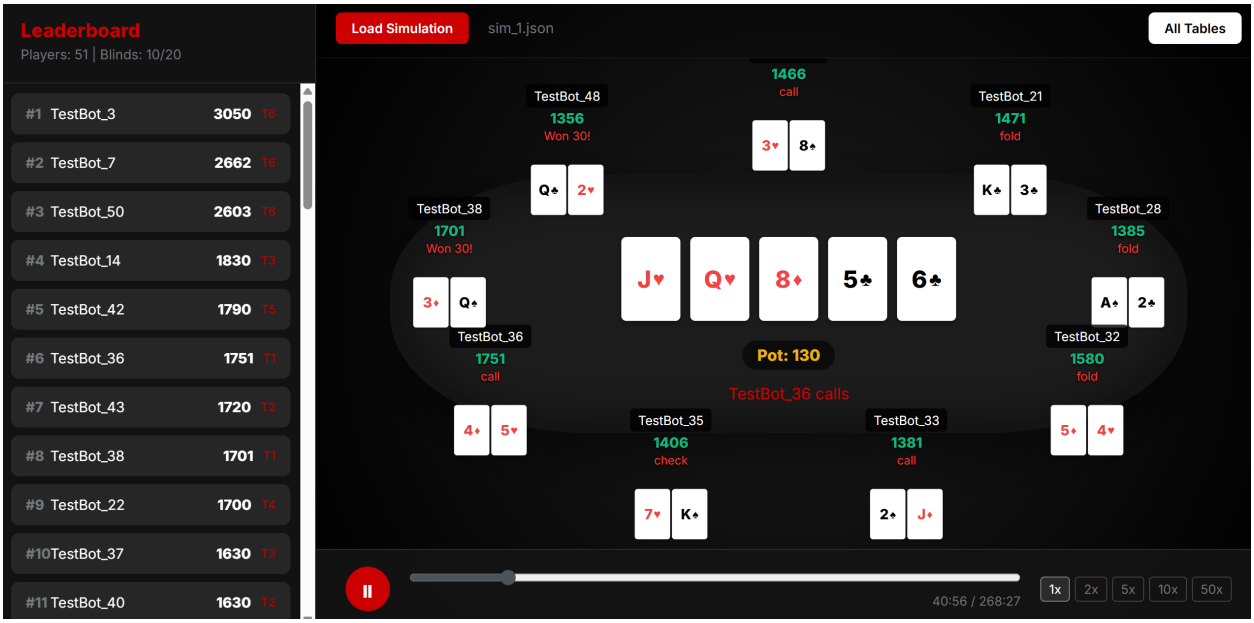


Figure 1: Screenshot of the MTT-Pokerbot-Engine visualizer, showing the leaderboard (left) and the current state of table T1 (right).

4. **Visualize:** Run `python server.py` and navigate to http://localhost:8000/visualizer. Figure 1 shows a screenshot of the visualizer interface, displaying the overall leaderboard and a detailed view of an ongoing table.